

TP 4: hadoop MapReduce

Université / École :

- Université Ibn Tofail, Kénitra

Encadrement pédagogique :

- Pr. Hajar Filali

Rédigé par :

- Mohammed Jebrou
- Hajar Farid

Année universitaire :

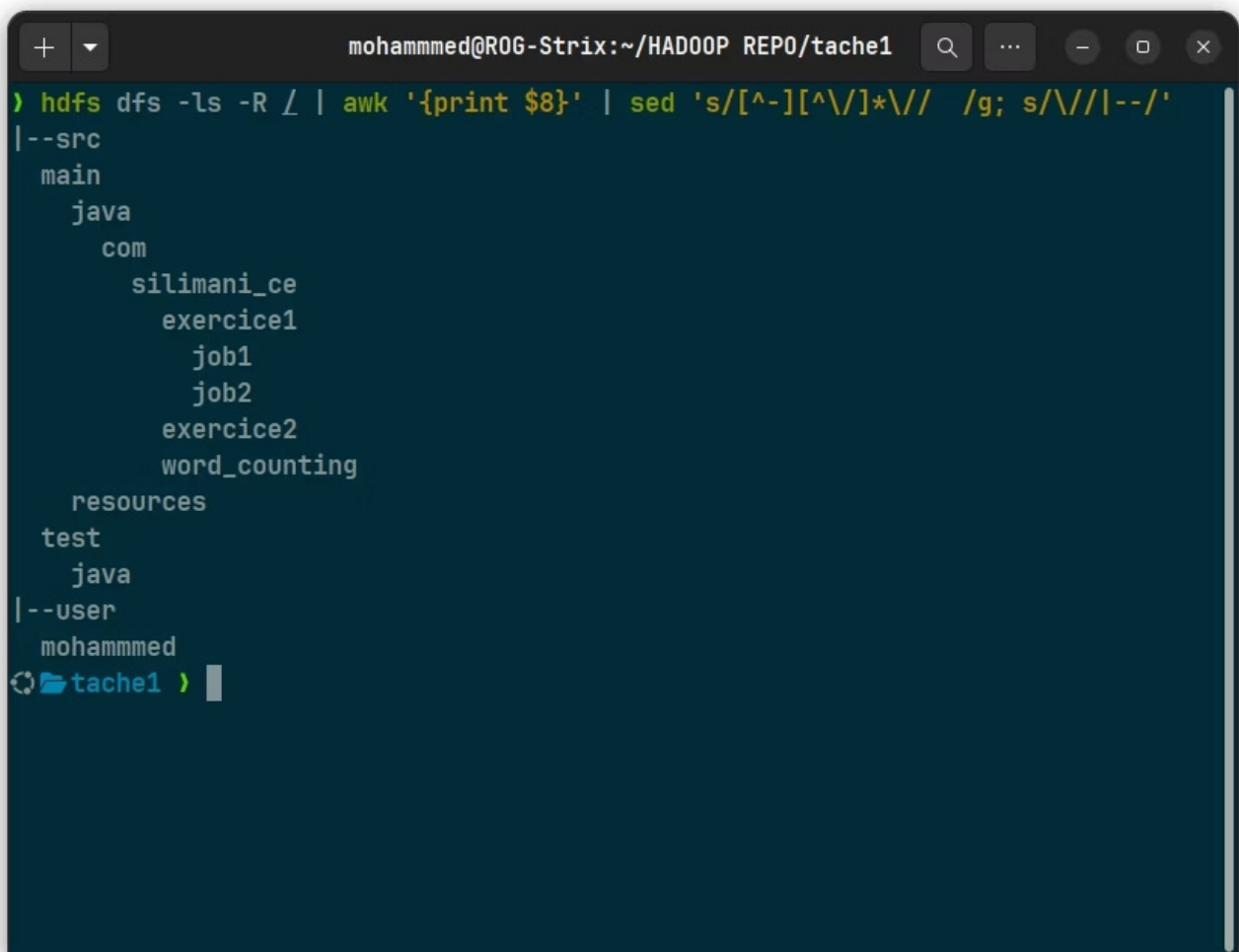
- 2025–2026

Préparation

1- Introduction

Ce travail a pour objectif de découvrir le modèle MapReduce en utilisant Hadoop. Nous avons réalisé plusieurs exercices pour analyser des données de ventes et des fichiers de logs. L'objectif est de comprendre le rôle du Mapper, du Reducer et du Driver dans le traitement distribué des données.

2. Preparation de structure du projet

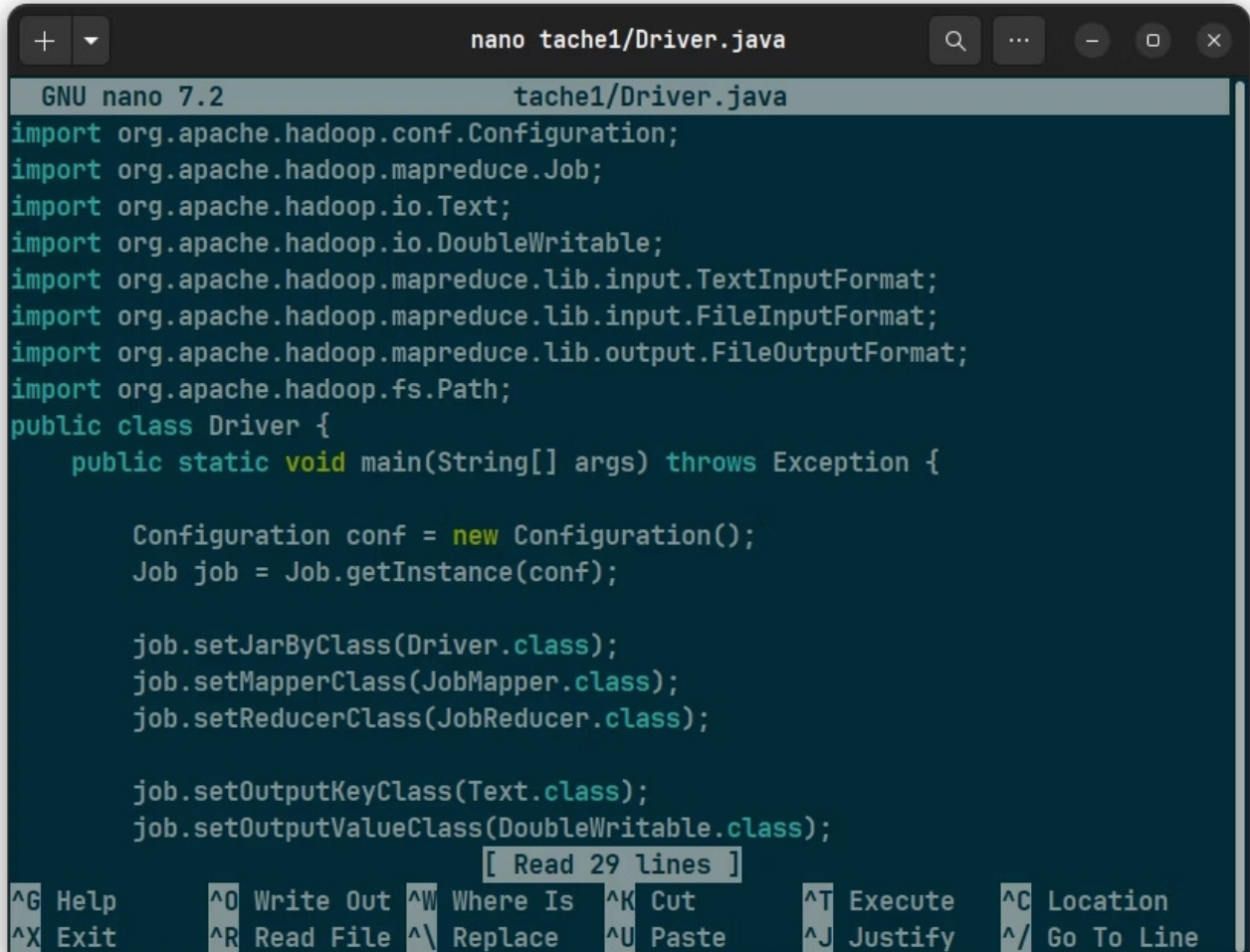


```
mohammed@ROG-Strix:~/HADOOP_REPO/tache1
> hdfs dfs -ls -R / | awk '{print $8}' | sed 's/[^-][^\\/]*/ /g; s/\\/|--/'
|--src
  main
    java
      com
        silimani_ce
          exercice1
            job1
            job2
          exercice2
          word_counting
  resources
  test
    java
  --user
  mohammed
tache1 >
```

Exercice 1

La tâche 1

1- code source du Driver



```
GNU nano 7.2 tache1/Driver.java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.fs.Path;
public class Driver {
    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf);

        job.setJarByClass(Driver.class);
        job.setMapperClass(JobMapper.class);
        job.setReducerClass(JobReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);
    }
}
```

[Read 29 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^/ Go To Line

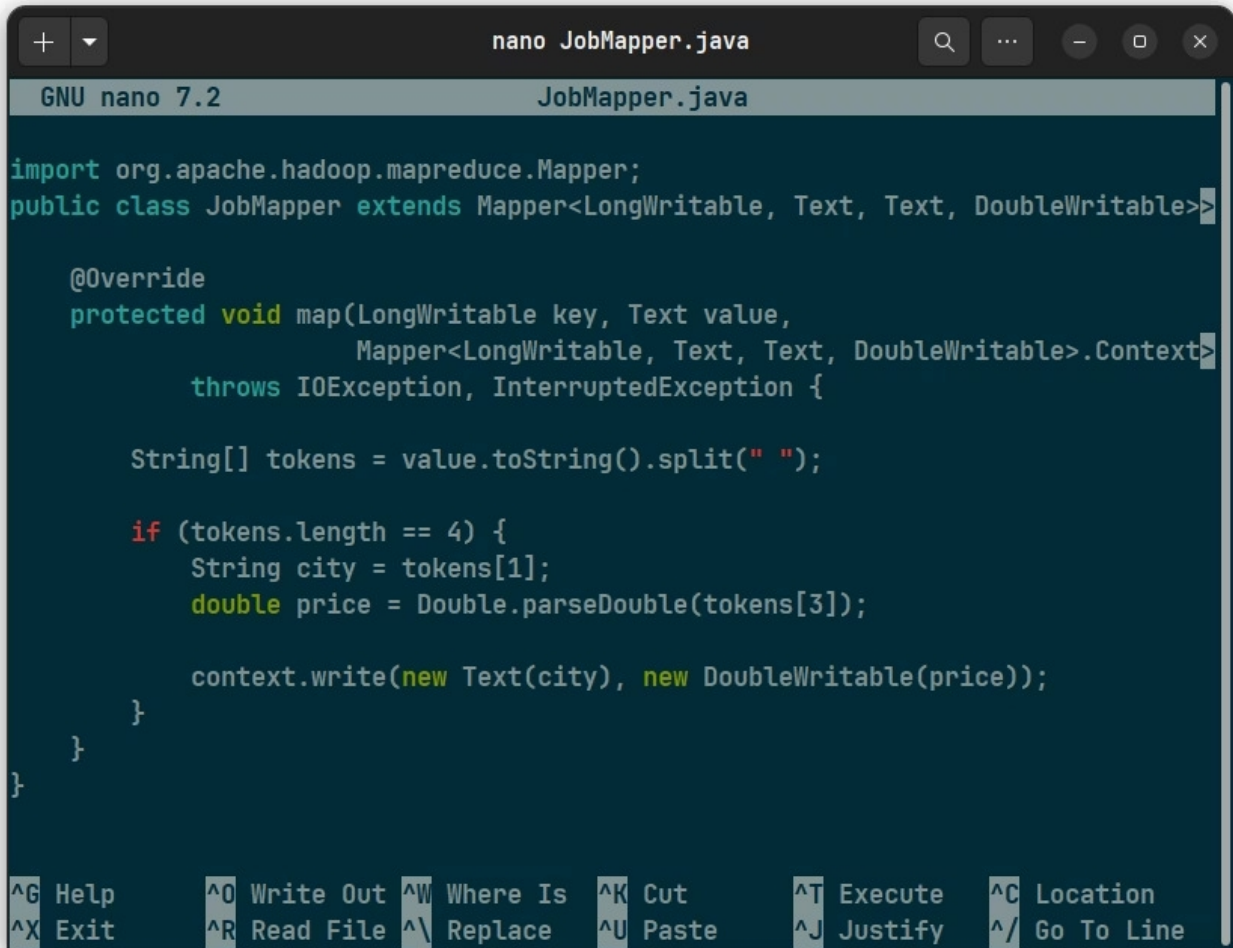
Le Driver est responsable de la configuration du job MapReduce.

Il définit le Mapper, le Reducer, les types de données, les chemins d'entrée/sortie et lance l'exécution du job.

Exercice 1

La tâche 1

2- source code du Mapper



```
import org.apache.hadoop.mapreduce.Mapper;
public class JobMapper extends Mapper<LongWritable, Text, Text, DoubleWritable>

    @Override
    protected void map(LongWritable key, Text value,
                        Mapper<LongWritable, Text, Text, DoubleWritable>.Context
                        throws IOException, InterruptedException {

        String[] tokens = value.toString().split(" ");

        if (tokens.length == 4) {
            String city = tokens[1];
            double price = Double.parseDouble(tokens[3]);

            context.write(new Text(city), new DoubleWritable(price));
        }
    }
}
```

Le Mapper sert à lire les données d'entrée ligne par ligne et à produire des paires (clé, valeur). Dans cet exercice, il extrait la ville et le prix afin de préparer les données pour le Reducer.

Exemple

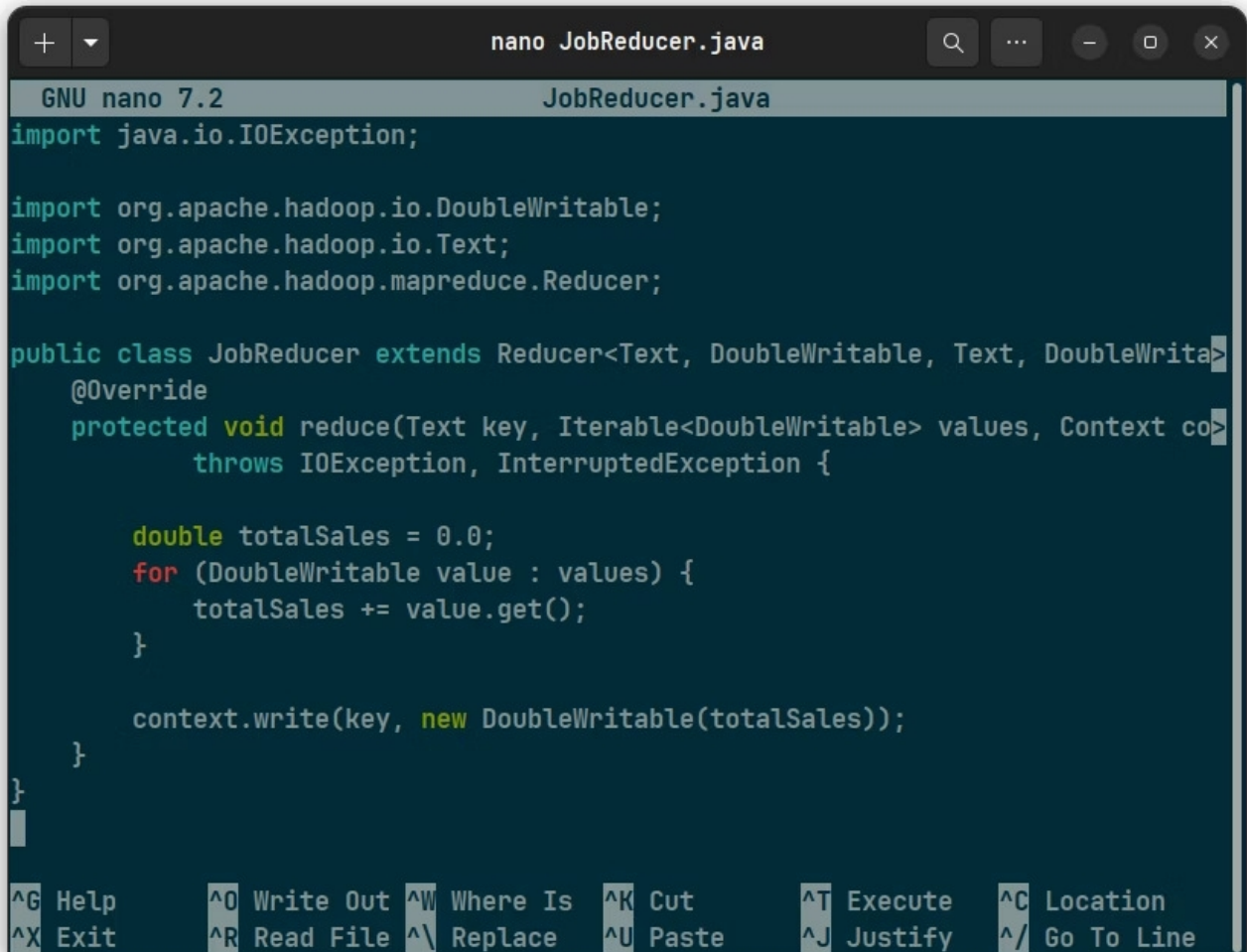
```
//le programme reçoit une ligne brute et effectue le fractionnement
tokens = ["2023-02-20", "Paris", "Product1", "95.0"]

city = tokens[1]→ Paris
price = tokens[3]→ 95.0
// produire une liste du (clé = ville ,valeur = prix)
(Paris, 95.0)
```

Exercice 1

La tâche 1

3- Source code du Reducer



```
GNU nano 7.2 JobReducer.java
import java.io.IOException;

import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class JobReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable>
    @Override
    protected void reduce(Text key, Iterable<DoubleWritable> values, Context context)
        throws IOException, InterruptedException {

        double totalSales = 0.0;
        for (DoubleWritable value : values) {
            totalSales += value.get();
        }

        context.write(key, new DoubleWritable(totalSales));
    }
}
```

Le Reducer reçoit les paires (clé, liste de valeurs) issues du Mapper et calcule une agrégation, ici la somme des ventes pour chaque ville, afin de produire le résultat final.

Exemple

```
//entrées
(Paris,95)
(Paris,85)
(Paris,100)
(London,120)
(London,90)
```

```
//agrégation
Paris → [95, 85, 100]
London → [120, 90]
```

```
//somme
Paris 280
London 210
```

Exercice 1

La tâche 1

4- Fichier vents.txt

```
mohammed@ROG-Strix:~/HAD00P_REPO/tache1
> hdfs dfs -cat /ventes.txt
2023-01-01 Paris Product1 100.0
2023-01-02 Paris Product2 75.0
2023-01-03 London Product1 120.0
2023-01-04 London Product2 90.0
2023-01-05 NewYork Product1 80.0
2023-01-06 NewYork Product2 70.0
2023-01-07 Paris Product1 95.0
2023-01-08 Paris Product2 85.0
2023-01-09 London Product1 110.0
2023-01-10 London Product2 100.0
tache1
```

5 - Chargement des fichiers dans HDFS

```
mohammed@ROG-Strix:~/HAD00P_REPO/tache1
> hdfs dfs -put {Driver,JobMapper,JobReducer}.java /src/main/java/com/silimani_ce/exercice1/job1/
> hdfs dfs -ls -R / | awk '{print $8}' | sed 's/[^-][^\\/]*/ /g; s/\\/|---/'
|--src
  main
    java
      com
        silimani_ce
          exercice1
            job1
              Driver.java
              JobMapper.java
              JobReducer.java
            job2
          exercice2
          word_counting
        resources
      test
        java
|--user
  mohammed
    ventes.txt
|--ventes.txt
tache1
```

Exercice 1

La tâche 1

6- Génération du MapReduceClient.jar

```
mohammed@ROG-Strix:~/HADOOP_REPO/tache1
> mkdir build
> javac -classpath "${hadoop classpath}" -d build Driver.java JobMapper.java Job
Reducer.java

> jar -cvf MapReduceClient.jar -C build .
added manifest
adding: Driver.class(in = 1315) (out= 690)(deflated 47%)
adding: JobMapper.class(in = 1692) (out= 670)(deflated 60%)
adding: JobReducer.class(in = 1608) (out= 665)(deflated 58%)
> ls
build      JobMapper.java  MapReduceClient.jar
Driver.java JobReducer.java  ventes.txt
tache1
```

Le fichier JAR est une archive contenant les classes compilées du programme MapReduce (Driver, Mapper, Reducer) du tâche 1 . Il est utilisé par Hadoop pour exécuter le job sur le cluster.

Exercice 1

La tâche 1

7-Execution du script

```
mohammed@ROG-Strix:~/HAD00P_REPO/tache1
> hadoop jar MapReduceClient.jar \
com.slimani_ce.exercice1.job1.Driver \
/ventes.txt /output
26/04/14 00:45:41 INFO Configuration.deprecation: session.id is deprecated. Instead, use dfs.metrics.session-id
26/04/14 00:45:41 INFO jvm.JvmMetrics: Initializing JVM Metrics with processName=JobTracker, sessionId=
26/04/14 00:45:41 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
26/04/14 00:45:41 INFO input.FileInputFormat: Total input paths to process : 1
26/04/14 00:45:41 INFO mapreduce.JobSubmitter: number of splits:1
26/04/14 00:45:41 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local959418295_0001
26/04/14 00:45:41 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
26/04/14 00:45:41 INFO mapreduce.Job: Running job: job_local959418295_0001
26/04/14 00:45:41 INFO mapred.LocalJobRunner: OutputCommitter set in config null
26/04/14 00:45:41 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 1
26/04/14 00:45:41 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
26/04/14 00:45:41 INFO mapred.LocalJobRunner: Waiting for map tasks
26/04/14 00:45:41 INFO mapred.LocalJobRunner: Starting task: attempt_local959418
```

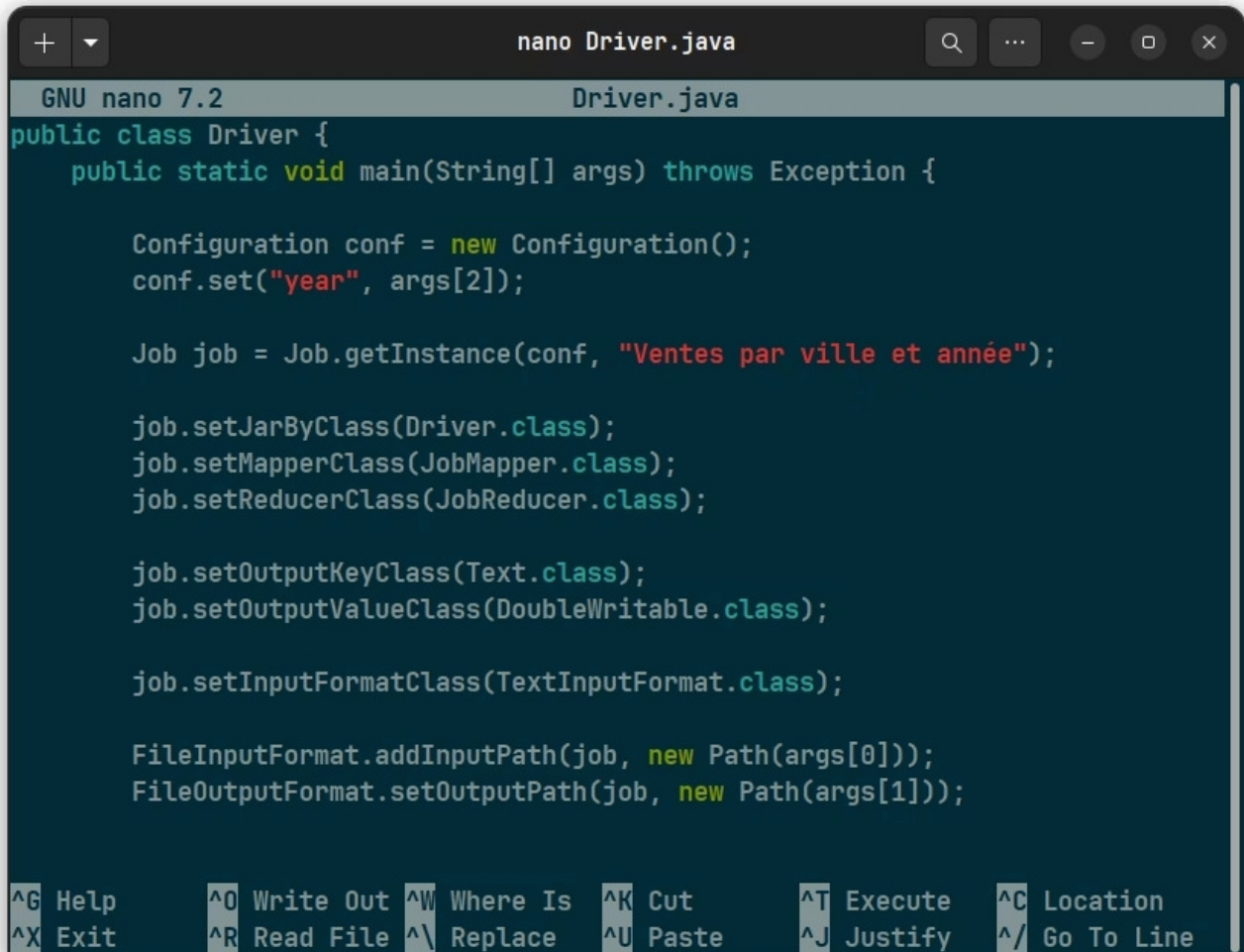
8- Verification du résultat

```
mohammed@ROG-Strix:~/HAD00P_REPO/tache1
> hdfs dfs -cat /output/part-r-00000
London 420.0
NewYork 150.0
Paris 355.0
tache1 >
```


Exercice 1

La tâche 2

1- code source de Driver



```
GNU nano 7.2 Driver.java
public class Driver {
    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        conf.set("year", args[2]);

        Job job = Job.getInstance(conf, "Ventes par ville et année");

        job.setJarByClass(Driver.class);
        job.setMapperClass(JobMapper.class);
        job.setReducerClass(JobReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        job.setInputFormatClass(TextInputFormat.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

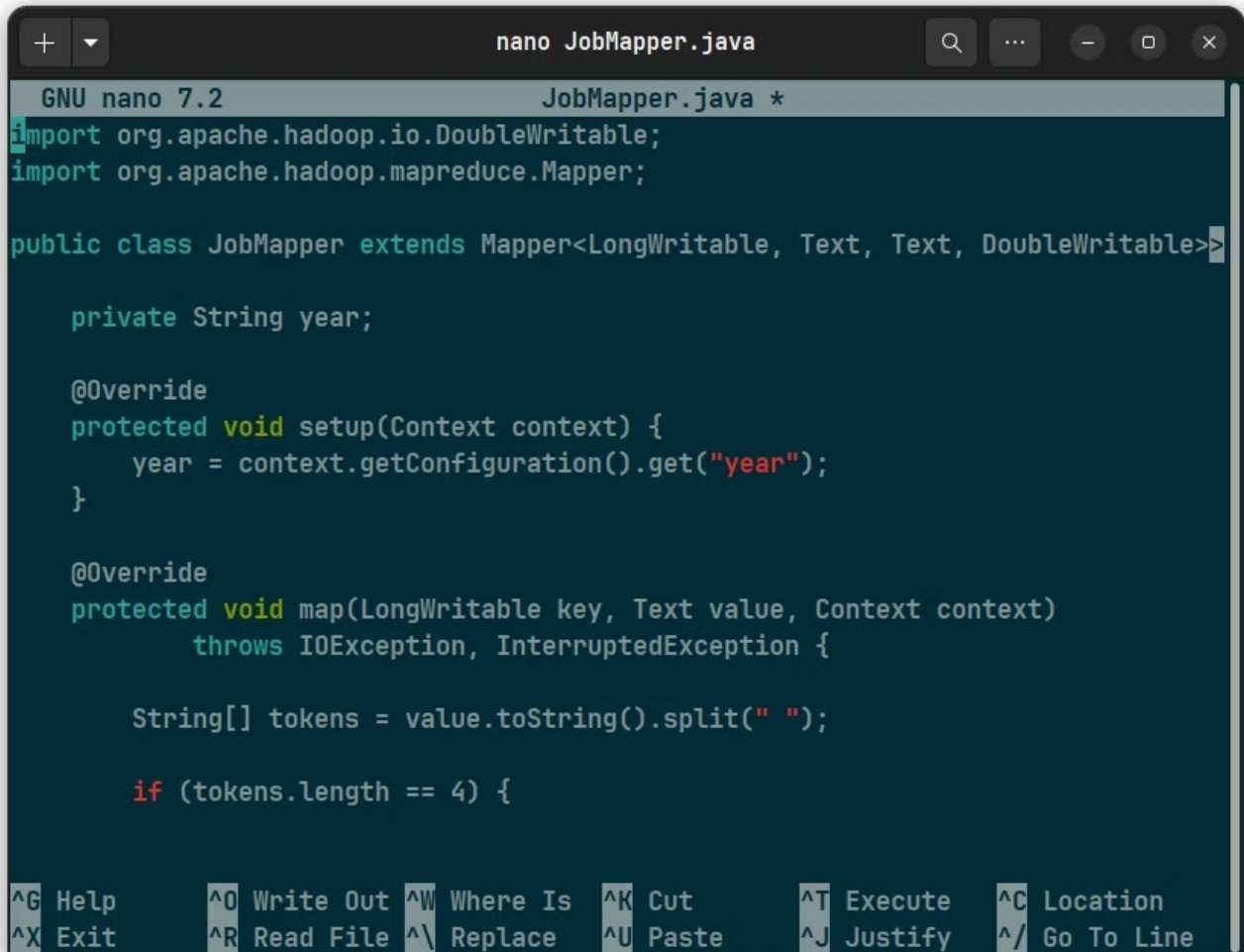
        ^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
        ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Le Driver est la classe principale d'un programme MapReduce. Il permet de configurer le job en définissant le Mapper, le Reducer, les types de données, ainsi que les chemins d'entrée et de sortie. Il lance ensuite l'exécution du job sur Hadoop

Exercice 1

La tâche 2

2- code source du Mapper



```
GNU nano 7.2 JobMapper.java *
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.mapreduce.Mapper;

public class JobMapper extends Mapper<LongWritable, Text, Text, DoubleWritable>

    private String year;

    @Override
    protected void setup(Context context) {
        year = context.getConfiguration().get("year");
    }

    @Override
    protected void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String[] tokens = value.toString().split(" ");

        if (tokens.length == 4) {
```

Le Mapper de la tâche 2 lit chaque ligne, extrait l'année à partir de la date, filtre les données selon l'année demandée, puis émet des paires (ville, prix) pour les lignes correspondantes.

Exemple

#entrées

2022 Paris 100
2023 Paris 95
2023 London 110

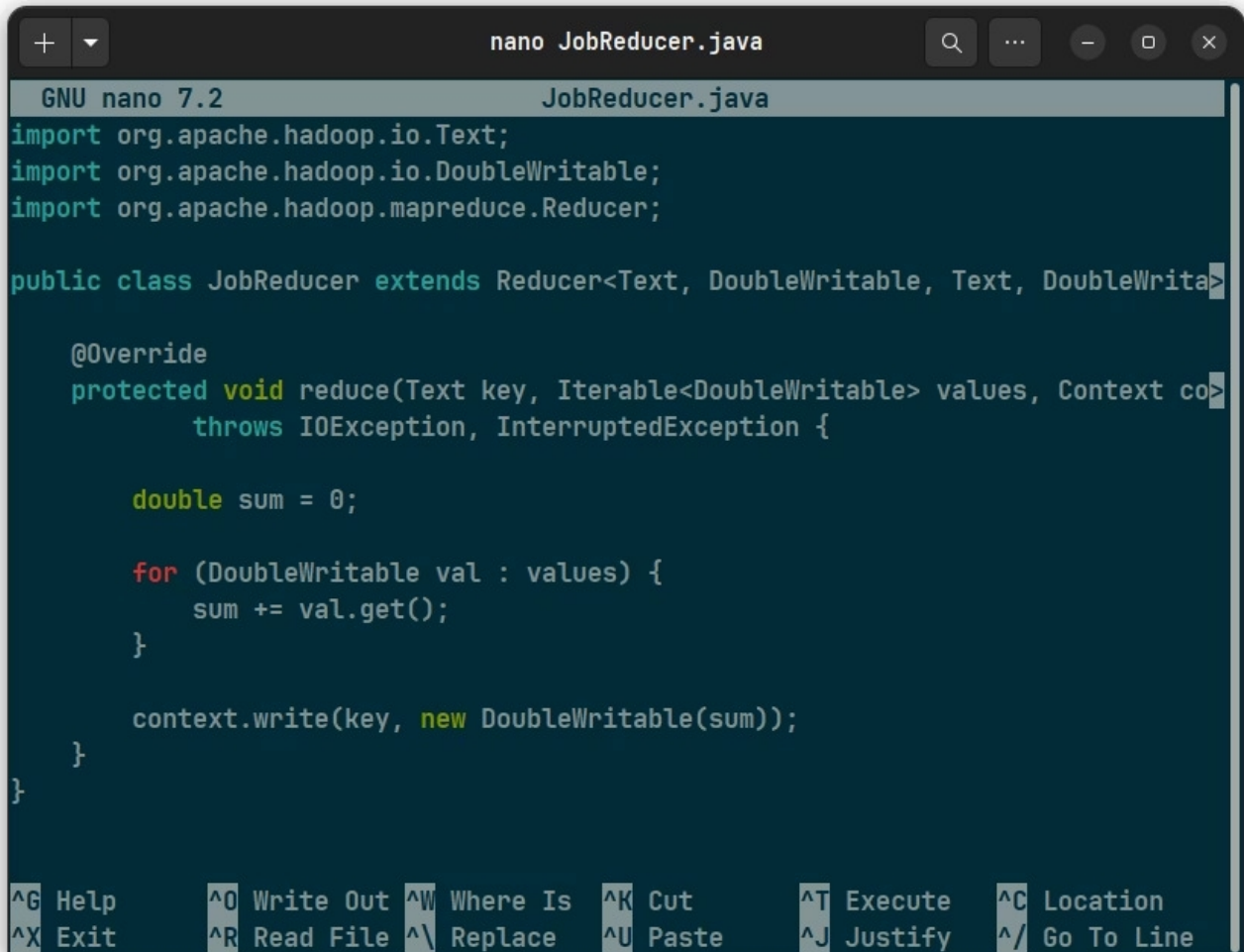
//sortie

(Paris,95)
(London,110)

Exercice 1

La tâche 2

3- code source de Reducer



```
GNU nano 7.2 JobReducer.java
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.mapreduce.Reducer;

public class JobReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    @Override
    protected void reduce(Text key, Iterable<DoubleWritable> values, Context context)
        throws IOException, InterruptedException {

        double sum = 0;

        for (DoubleWritable val : values) {
            sum += val.get();
        }

        context.write(key, new DoubleWritable(sum));
    }
}
```

Help Write Out Where Is Cut Execute Location
Exit Read File Replace Paste Justify Go To Line

Le Reducer est une fonction qui reçoit les paires (clé, liste de valeurs) issues du Mapper et applique une opération d'agrégation (comme la somme) afin de produire le résultat final.

//entrées
(Paris,95)
(Paris,85)
(London,110)
(London,100)
(NewYork,70)

//agrégation
Paris → [95, 85]
London → [110, 100]
NewYork → [70]

//somme
Paris 180
London 210
NewYork 70

Exercice 1

La tâche 2

4-creation du MapReduceClient1.jar

```
mohammed@ROG-Strix:~/HAD00P_REPO/tache2
> mkdir build
> javac -classpath "${hadoop classpath}" -d build *.java
> jar -cvf MapReduceClient1.jar -C build/ .
added manifest
adding: com/(in = 0) (out= 0)(stored 0%)
adding: com/slimani_ce/(in = 0) (out= 0)(stored 0%)
adding: com/slimani_ce/exercice1/(in = 0) (out= 0)(stored 0%)
adding: com/slimani_ce/exercice1/job2/(in = 0) (out= 0)(stored 0%)
adding: com/slimani_ce/exercice1/job2/Driver.class(in = 1635) (out= 862)(deflated 47%)
adding: com/slimani_ce/exercice1/job2/JobMapper.class(in = 2311) (out= 876)(deflated 62%)
adding: com/slimani_ce/exercice1/job2/JobReducer.class(in = 1638) (out= 690)(deflated 57%)
tache2 >
```

Le Reducer reçoit les paires (clé, liste de valeurs) issues du Mapper.

Il applique une opération d'agrégation (comme la somme) pour produire le résultat final.

5-creation du ventes1.txt

```
mohammed@ROG-Strix:~/HAD00P_REPO/tache2
> hdfs dfs -cat /ventes1.txt
2022-01-01 Paris Product1 100.0
2022-02-15 Paris Product2 75.0
2022-03-20 London Product1 120.0
2022-04-12 London Product2 90.0
2022-05-02 NewYork Product1 80.0
2023-01-06 NewYork Product2 70.0
2023-02-20 Paris Product1 95.0
2023-03-18 Paris Product2 85.0
2023-04-25 London Product1 110.0
2023-05-05 London Product2 100.0
tache2 >
```


Exercice 1

La tâche 2

6-Execution du script

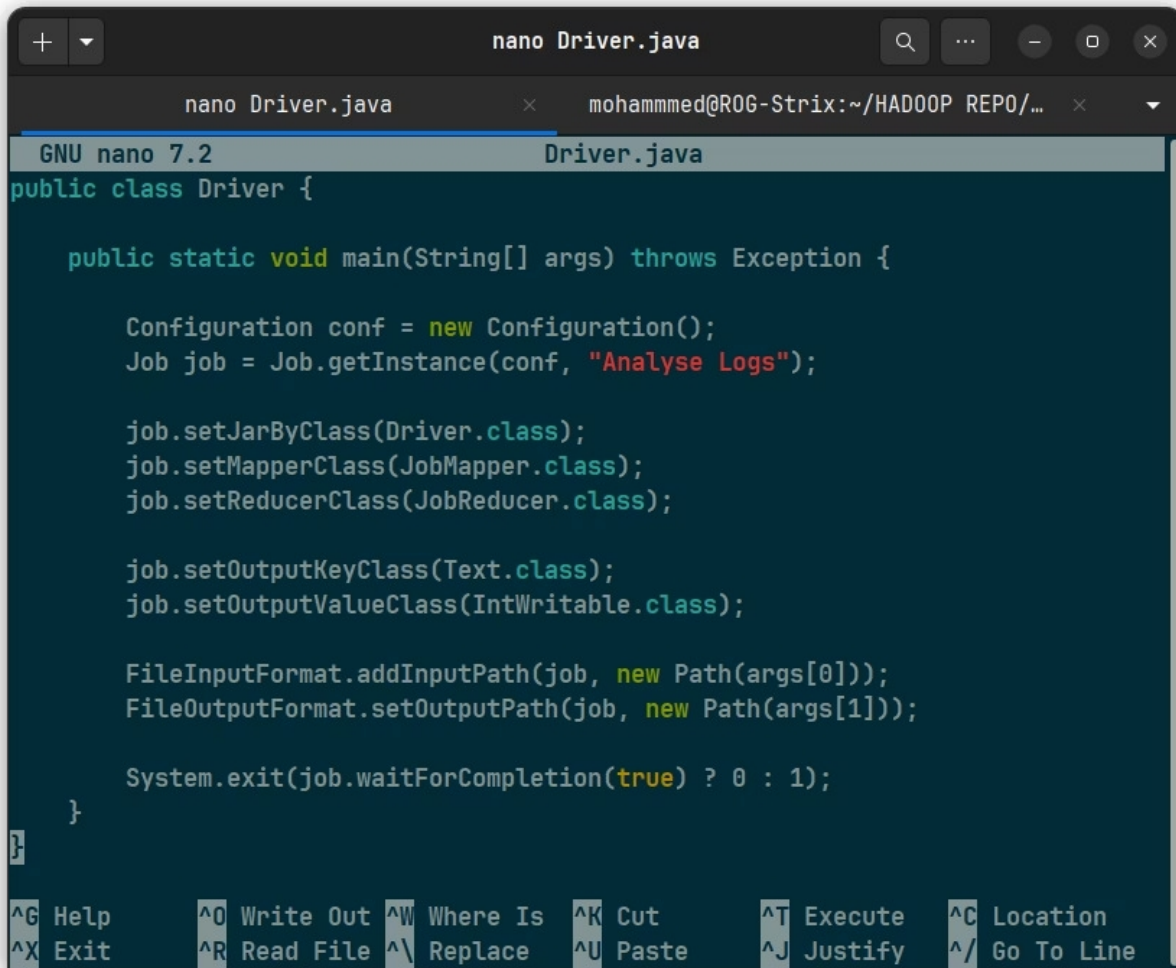
```
mohammed@ROG-Strix:~/HAD00P_REPO/tache2
> hadoop jar MapReduceClient1.jar \
com.slimani_ce.exercice1.job2.Driver \
/ventes1.txt /ex1_job2_output 2023
26/04/14 13:32:30 INFO Configuration.deprecation: session.id is deprecated. Instead, use dfs.metrics.session-id
26/04/14 13:32:30 INFO jvm.JvmMetrics: Initializing JVM Metrics with processName=JobTracker, sessionId=
26/04/14 13:32:30 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
26/04/14 13:32:30 INFO input.FileInputFormat: Total input paths to process : 1
26/04/14 13:32:30 INFO mapreduce.JobSubmitter: number of splits:1
26/04/14 13:32:30 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local243869962_0001
26/04/14 13:32:30 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
26/04/14 13:32:30 INFO mapreduce.Job: Running job: job_local243869962_0001
26/04/14 13:32:30 INFO mapred.LocalJobRunner: OutputCommitter set in config null
26/04/14 13:32:30 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 1
26/04/14 13:32:30 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
26/04/14 13:32:30 INFO mapred.LocalJobRunner: Waiting for map tasks
26/04/14 13:32:30 INFO mapred.LocalJobRunner: Starting task: attempt_local243869
```

7- Verification des resultat

```
mohammed@ROG-Strix:~/HAD00P_REPO/tache2
> hdfs dfs -cat /ex1_job2_output/part-r-00000
London 210.0
NewYork 70.0
Paris 180.0
tache2 >
```

Exercice 2

1-Source code du Driver



```
GNU nano 7.2 Driver.java
public class Driver {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Analyse Logs");

        job.setJarByClass(Driver.class);
        job.setMapperClass(JobMapper.class);
        job.setReducerClass(JobReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

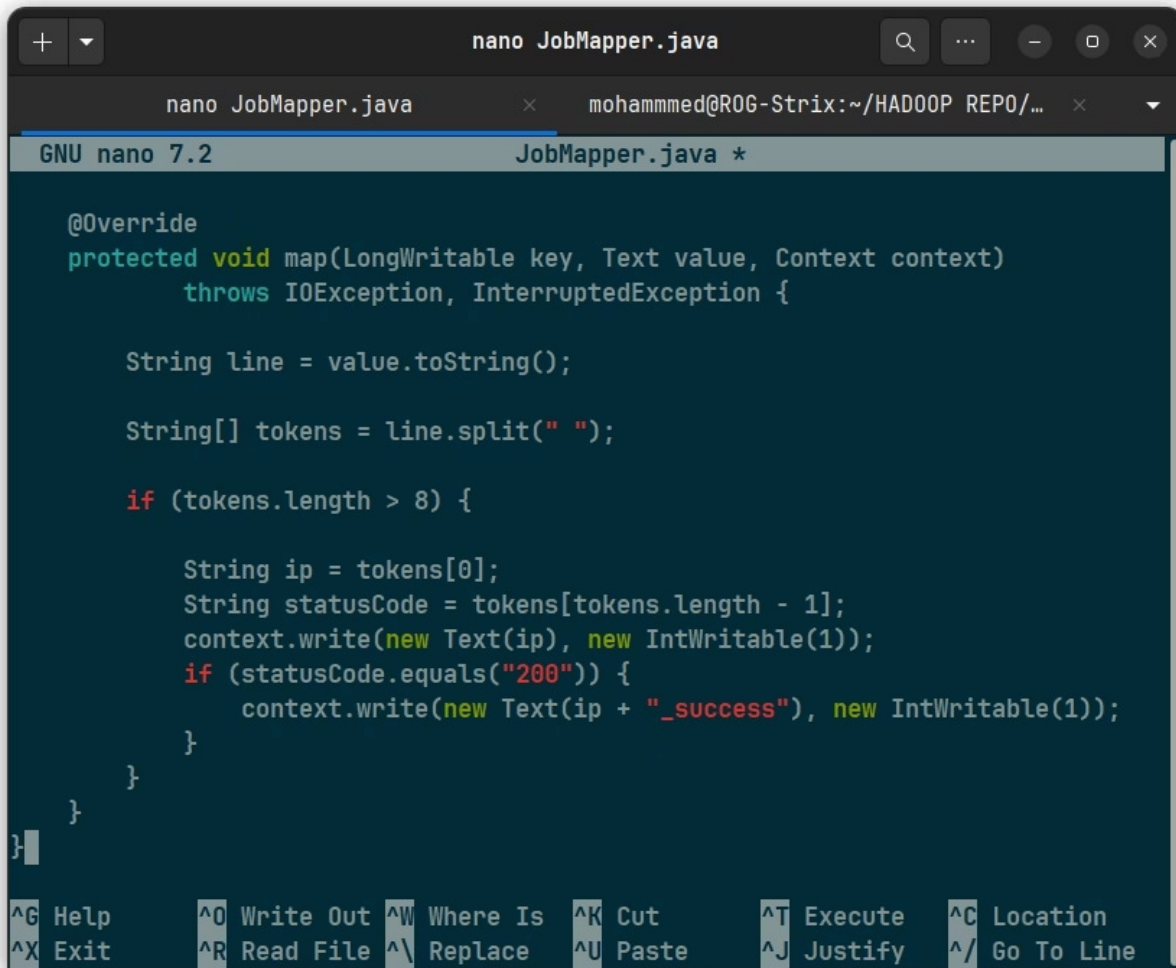
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

Le Driver configure le job MapReduce en définissant les classes Mapper et Reducer, les types de sortie ainsi que les chemins d'entrée et de sortie, puis lance l'exécution du traitement.

Exercice 2

2-Source code du Mapper



```
nano JobMapper.java
GNU nano 7.2 JobMapper.java *

@Override
protected void map(LongWritable key, Text value, Context context)
    throws IOException, InterruptedException {

    String line = value.toString();

    String[] tokens = line.split(" ");

    if (tokens.length > 8) {

        String ip = tokens[0];
        String statusCode = tokens[tokens.length - 1];
        context.write(new Text(ip), new IntWritable(1));
        if (statusCode.equals("200")) {
            context.write(new Text(ip + "_success"), new IntWritable(1));
        }
    }
}
}
```

Le Mapper lit chaque ligne du fichier de logs, extrait l'adresse IP et le code HTTP, puis émet une paire (IP, 1) uniquement si le code est 200 afin de compter les requêtes réussies.

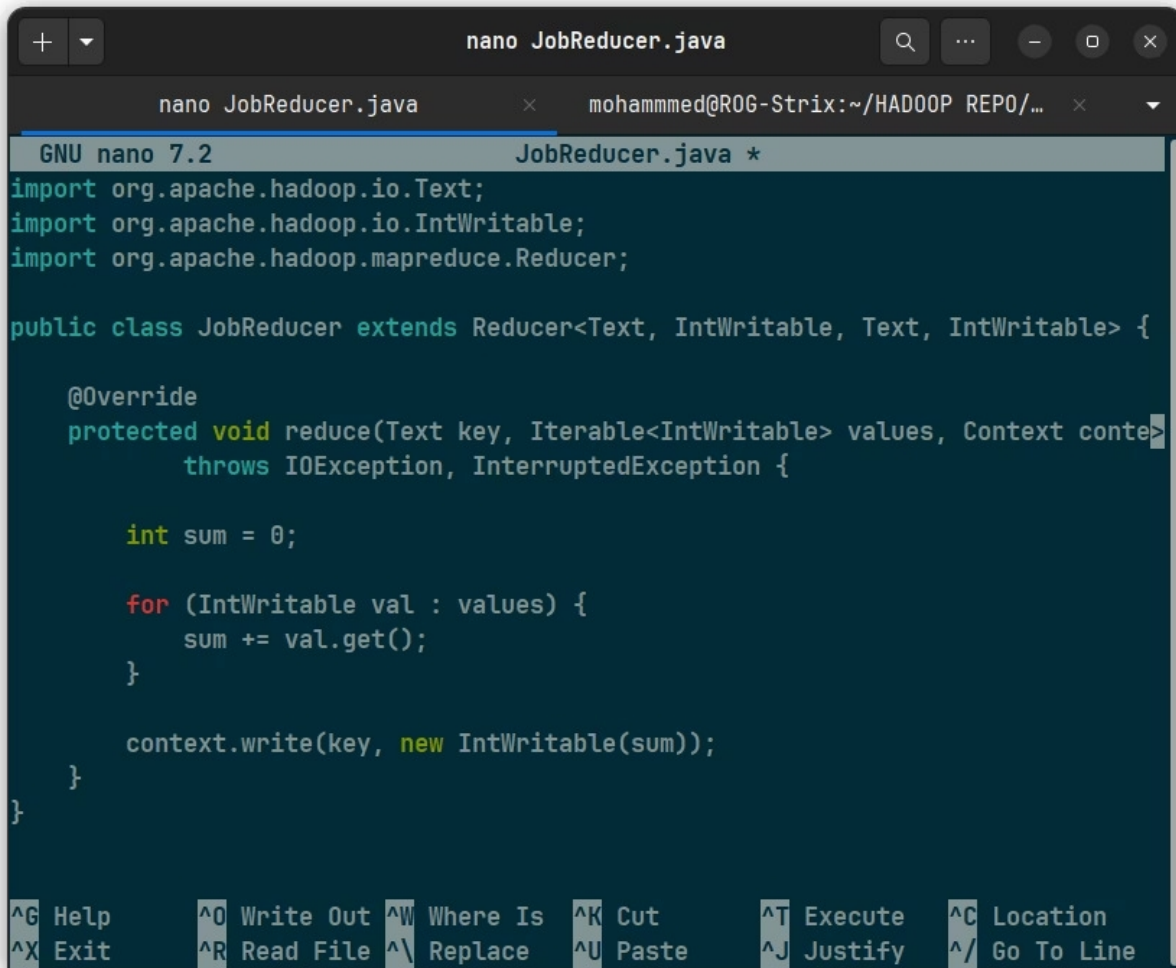
Exemple

```
//entrées
192.168.1.1 .. HTTP/1.1" 200 1234
192.168.1.2 .. HTTP/1.1" 200 567
192.168.1.3 .. HTTP/1.1" 200 5674
192.168.1.1 .. HTTP/1.1" 200 8757
192.168.1.1 .. HTTP/1.1" 404 7954
```

```
//sortie
(192.168.1.1, 1)
(192.168.1.2, 1)
(192.168.1.3, 1)
(192.168.1.1, 1)
//ce ligne sera ignorer
```

Exercice 2

3-Source code du Reducer



```
GNU nano 7.2 JobReducer.java *
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.Reducer;

public class JobReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {
            sum += val.get();
        }

        context.write(key, new IntWritable(sum));
    }
}
```

Le Reducer reçoit pour chaque clé (IP) une liste de valeurs (1) issues du Mapper et calcule leur somme afin d'obtenir le nombre de requêtes réussies par adresse IP

Exemple

```
//entrées
(192.168.1.1, 1)
(192.168.1.2, 1)
(192.168.1.3, 1)
(192.168.1.1, 1)
```

```
//agrégation
192.168.1.1 → [1,1]
192.168.1.2 → [1]
192.168.1.3 → [1]
```

```
#somation
192.168.1.1 → 2
192.168.1.2 → 1
192.168.1.3 → 1
```

Exercice 2

4-creation du fichier MapReduceLogs.jar

```
mohammed@ROG-Strix:~/HADOOP_REPO/TP1/exercice2
mohammed@ROG-Strix:~/HADOOP_REPO/... x mohammed@ROG-Strix:~/HADOOP_REPO/... x
> mkdir build
> javac -classpath "$(hadoop classpath)" -d build *.java
> jar -cvf MapReduceLogs.jar -C build/ .
added manifest
adding: com/(in = 0) (out= 0)(stored 0%)
adding: com/slimani_ce/(in = 0) (out= 0)(stored 0%)
adding: com/slimani_ce/exercice2/(in = 0) (out= 0)(stored 0%)
adding: com/slimani_ce/exercice2/Driver.class(in = 1421) (out= 774)(deflated 45%)
)
adding: com/slimani_ce/exercice2/JobMapper.class(in = 1882) (out= 780)(deflated 58%)
adding: com/slimani_ce/exercice2/JobReducer.class(in = 1615) (out= 685)(deflated 57%)
> ls
build  Driver.java  JobMapper.java  JobReducer.java  MapReduceLogs.jar
exercice2 )
```

5- Verification du Logs.txt

```
mohammed@ROG-Strix:~/HADOOP_REPO/TP1/exercice2
mohammed@ROG-Strix:~/HADOOP_REPO/... x mohammed@ROG-Strix:~/HADOOP_REPO/... x
> hdfs dfs -cat /Logs.txt
192.168.1.1 - - [12/May/2023:15:30:45 +0000] "GET /page1 HTTP/1.1" 200 1234
192.168.1.2 - - [12/May/2023:15:31:02 +0000] "GET /page2 HTTP/1.1" 404 567
192.168.1.1 - - [12/May/2023:15:32:10 +0000] "GET /page1 HTTP/1.1" 200 789
192.168.1.3 - - [12/May/2023:15:32:35 +0000] "GET / HTTP/1.1" 200 987
192.168.1.1 - - [12/May/2023:15:33:12 +0000] "GET /page2 HTTP/1.1" 200 1200
192.168.1.2 - - [12/May/2023:15:33:55 +0000] "GET /page1 HTTP/1.1" 404 800
192.168.1.3 - - [12/May/2023:15:34:20 +0000] "GET /page2 HTTP/1.1" 200 1600
192.168.1.1 - - [12/May/2023:15:34:48 +0000] "GET /page1 HTTP/1.1" 200 600
exercice2 )
```

Exercice 2

6-Execution de script Jar

```
mohammed@ROG-Strix:~/HADOOP_REPO/TP1/exercice2
mohammed@ROG-Strix:~/HADOOP_REPO/... x mohammed@ROG-Strix:~/HADOOP_REPO/... x
> hadoop jar MapReduceLogs.jar \
com.slimani_ce.exercice2.Driver \
/Logs.txt /ex2_output
26/04/14 14:32:14 INFO Configuration.deprecation: session.id is deprecated. Instead, use dfs.metrics.session-id
26/04/14 14:32:14 INFO jvm.JvmMetrics: Initializing JVM Metrics with processName=JobTracker, sessionId=
26/04/14 14:32:14 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
26/04/14 14:32:14 INFO input.FileInputFormat: Total input paths to process : 1
26/04/14 14:32:14 INFO mapreduce.JobSubmitter: number of splits:1
26/04/14 14:32:14 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local339726072_0001
26/04/14 14:32:14 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
26/04/14 14:32:14 INFO mapreduce.Job: Running job: job_local339726072_0001
26/04/14 14:32:14 INFO mapred.LocalJobRunner: OutputCommitter set in config null
26/04/14 14:32:14 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 1
26/04/14 14:32:14 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
26/04/14 14:32:14 INFO mapred.LocalJobRunner: Waiting for map tasks
26/04/14 14:32:14 INFO mapred.LocalJobRunner: Starting task: attempt_local339726
```

7- Verification du résultat

```
mohammed@ROG-Strix:~/HADOOP_REPO/TP1/exercice2
mohammed@ROG-Strix:~/HADOOP_REPO/... x mohammed@ROG-Strix:~/HADOOP_REPO/... x
> hdfs dfs -cat /ex2_output/part-r-00000
192.168.1.1 4
192.168.1.3 2
exercice2 >
```

Conclusion

Ce TP nous a permis d'aborder concrètement le fonctionnement de **MapReduce** à travers trois exercices progressifs. Le premier a facilité la prise en main du principe de découpage et de traitement distribué des données. Le deuxième a montré l'exécution d'un script JAR et la vérification du résultat obtenu, ce qui a permis de mieux comprendre le déroulement d'un traitement Hadoop. Enfin, le troisième exercice a renforcé la maîtrise de l'ensemble du processus en mettant en pratique les étapes nécessaires à l'analyse et à l'exploitation des données.

De manière générale, **MapReduce** constitue un modèle essentiel pour le traitement de grandes quantités de données réparties sur plusieurs machines. Il permet de paralléliser les calculs, d'améliorer les performances et de gérer efficacement des volumes de données importants tout en assurant une bonne scalabilité. Ce paradigme est aujourd'hui largement utilisé dans les environnements de big data.

Grâce à ce TP, nous avons acquis des compétences pratiques en traitement distribué, en utilisation d'outils Hadoop, en exécution de scripts JAR et en interprétation des résultats. Nous avons également renforcé notre compréhension des étapes d'un traitement MapReduce, ainsi que de son intérêt dans les architectures de calcul à grande échelle.